

Corso di Calcolo Parallelo

Laurea Magistrale in Informatica—Università di Bologna

Prova scritta, 25 Maggio 2026

- La prova dura 90 minuti
- Durante la prova non è consentito consultare libri, appunti o altro materiale.
- Non è consentito l'uso di dispositivi elettronici (ad esempio, cellulari, tablet...), né interagire in alcun modo con gli altri ammessi alla prova pena l'esclusione, che potrà avvenire anche dopo il termine della stessa.
- Le risposte devono essere scritte **a penna** su questi fogli, in modo **leggibile**. Le parti illeggibili o scritte a matita saranno ignorate.
- Eventuali altri fogli possono essere utilizzati per la brutta copia, vanno consegnati ma non verranno valutati.
- I voti saranno pubblicati su AlmaEsami.
- I voti della prova scritta e del progetto restano validi fino alla sessione d'esame di **febbraio 2027** inclusa. Dopo tale data tutti i voti in sospeso saranno persi.

NON SCRIVERE NELLA TABELLA SOTTOSTANTE

D. 1	D. 2	D. 3	D. 4	D. 5
/ 6	/ 6	/ 6	/ 6	/ 6

Domanda 0.

Scrivere nome, cognome e numero di matricola in cima al primo foglio.

Domanda 1.

Dopo aver brevemente introdotto il concetto di architettura parallela a memoria distribuita, descriverne vantaggi e svantaggi motivando la risposta. Se lo si ritiene utile, è possibile fare riferimento ad esempi e/o applicazioni viste a lezione.

Risposta.

Domanda 2.

Discutere se e come sia possibile parallelizzare il frammento di codice seguente usando OpenMP. Solo nel caso il codice possa essere parallelizzato, evidenziare eventuali aspetti positivi e negativi della soluzione proposta.

```
#define N ...
double A[N][N];

/* ... */

for (int k=0; k<500; k++) {
    for (int i=1; i<N; i++) {
        for (int j=1; j<N-1; j++) {
            A[i][j] = A[i][j] + (A[i-1][j-1] + A[i-1][j] + A[i-1][j+1])/3.0;
        }
    }
}
```

Risposta.

Domanda 3.

È dato un array $v[N]$ di lunghezza N contenente valori interi appartenenti all'insieme $\{0, \dots, 99\}$; l'array non è ordinato. Si vuole calcolare l'istogramma $h[100]$ del numero di occorrenze di ciascun valore in $v[]$. In altre parole, per ogni $k = 0, \dots, 99$, $h[k]$ deve essere il numero di volte in cui il valore k compare in $v[]$. Descrivere un algoritmo in grado di risolvere il problema su architetture a memoria distribuita usando MPI.

Si assuma che:

- la lunghezza N di $v[]$ sia multipla del numero P di processi MPI, e sia molto maggiore di P ;
- il valore di N sia inizialmente noto a tutti i processi;
- il contenuto di $v[]$ sia inizialmente noto solo al processo 0;
- al termine dell'esecuzione il contenuto dell'istogramma $h[]$ debba essere noto al processo 0.

Si descriva nel modo più preciso possibile il comportamento di ciascun processo MPI. **Non è richiesto scrivere codice:** è possibile usare pseudocodice e/o linguaggio naturale, purché **la spiegazione sia il più precisa possibile**. In particolare, si indichi quale/i funzioni MPI si userebbero nei vari passi, e quali buffer di memoria è necessario usare/allocare.

Risposta.

Domanda 4.

1. Descrivere un problema che, a proprio giudizio, **si presti** ad essere risolto in modo efficiente su una GPU; motivare la risposta.
2. Descrivere un problema che, a proprio giudizio, **non si presti** ad essere risolto in modo efficiente su una GPU; motivare la risposta.

È possibile fare riferimento ad esempi e/o applicazioni visti a lezione, oppure descriverne di nuovi.

Risposta.

Domanda 5.

La funzione seguente usa un approccio divide-et-impera per calcolare il valore massimo presente nella porzione di un array $v[]$ compresa tra gli indici i e j , estremi inclusi.

```
int arraymax(const int *v, int i, int j)
{
    if (i > j)
        return INT_MIN; /* minimo valore rappresentabile dal tipo "int". */
    else if (i == j)
        return v[i];
    else {
        const int m = (i+j)/2;
        int l, r;
        #pragma omp task shared(v, i, m, l)
        l = arraymax(v, i, m);
        #pragma omp task shared(v, j, m, r)
        r = arraymax(v, m+1, j);
        #pragma omp taskwait
        return (l > r ? l : r);
    }
}
```

La funzione viene inizialmente invocata da un singolo thread in una regione parallela con la chiamata $x = \text{arraymax}(v, 0, n-1)$.

Determinare *work*, *span* e *speedup* della funzione `arraymax()` applicata ad un array di lunghezza n , supponendo di avere $P = n$ processori.

Risposta.